RAG, Tested

Lesson 8 of 12 · From documents to cited answers, with a real retrieval test.

In this lesson → ① The seven steps ② Hybrid search and RRF ③ My retrieval test ④ Chunk size, measured ⑤ The fusion surprise ⑥ Failure points ⑦ Interview Q&A
Course project: ShopNest Copilot, the AI support assistant we move to the cloud, step by step.

1 RAG: an open-book exam

> A model only knows its training data. **RAG** looks up the relevant pages when a question arrives, puts them in the prompt, and the model answers from them, ideally quoting them.

2 The seven steps



Mnemonic: **P**retty **C**ats **C**an **E**at **H**ot **R**ed **G**rapes = Parse, Chunk, Context, Embed, Hybrid, Rerank, Generate.

Step	Notes
1 Parse	layout-aware: a flattened table gives wrong numbers. Keep metadata (source, date, tenant, who may read it)
2 Chunk	about 300 to 800 tokens with overlap; fixed, paragraph, semantic or parent-child
3 Context	say where each chunk came from (contextual retrieval)
4 Embed	once, offline; cache by text hash; record the model version

3 Steps 5 to 7

Step	Notes
5 Hybrid	vectors catch paraphrases, BM25 keywords catch exact terms (error codes, names); fuse the lists
6 Rerank	retrieve about 150, rerank with a cross-encoder, keep about 20. Too much context distracts: models skim the middle
7 Generate	chunk IDs; "answer only from the sources, cite the IDs, say not found if absent"

4 Reciprocal rank fusion

> **score(d) = Σ over lists of 1 ÷ (60 + rank)**: ranks only, no score scaling

Document	Calculation	Score
#1 vector and #3 keyword	1/61 + 1/63	**0.0323**
#2 vector only	1/62	0.0161

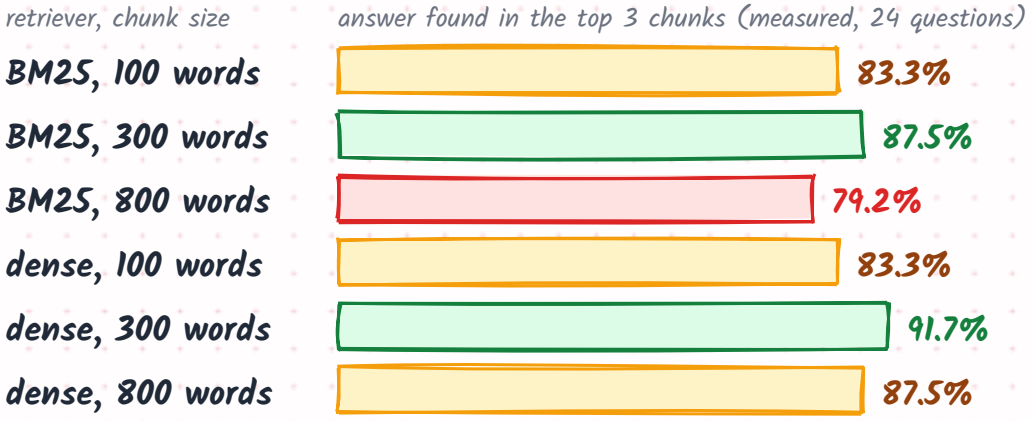
Found by both wins. Rewriting the question first (a hypothetical answer, or splitting it into parts) costs one extra model call.

5 My retrieval test

Item	Value
corpus	this course's notebook: **45,060 words**
questions	24 written by hand, each with the exact answer phrase
hit	a chunk contains the answer phrase; hit@k = answer in the top k chunks
BM25	keyword scoring (rank_bm25)
dense	**stand-in**: TF-IDF + 200-dim SVD (LSA), cosine. Not a neural embedding model
hybrid	reciprocal rank fusion of the two
chunks	N words, 15% overlap

Scripts: l08_rag_eval.py, l08_context_prefix.py, *corpus in* data/notebook_text.txt.

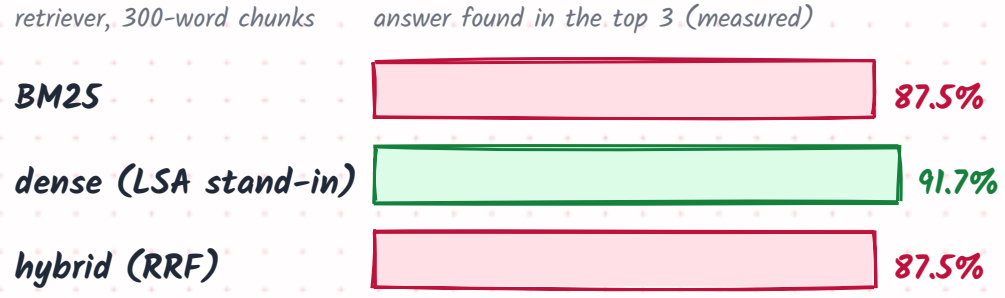
6 Chunk size, measured



Chunk	Method	hit@1	hit@3	hit@5
100 words	BM25	62.5%	83.3%	87.5%
100 words	dense	62.5%	83.3%	95.8%
300 words	BM25	62.5%	**87.5%**	91.7%
300 words	dense	66.7%	**91.7%**	95.8%
800 words	BM25	58.3%	79.2%	91.7%
800 words	dense	66.7%	87.5%	87.5%

Too small loses surroundings ("k = 60" alone means little); too big dilutes the match. 300 words was best for both.

7 The fusion surprise



the fancy fusion did not win here

300-word chunks	hit@1	hit@3	hit@5
BM25	62.5%	87.5%	91.7%
dense (stand-in)	66.7%	**91.7%**	95.8%
hybrid RRF	62.5%	87.5%	91.7%

both found it	only BM25	only dense	neither
21 questions	0	1	2

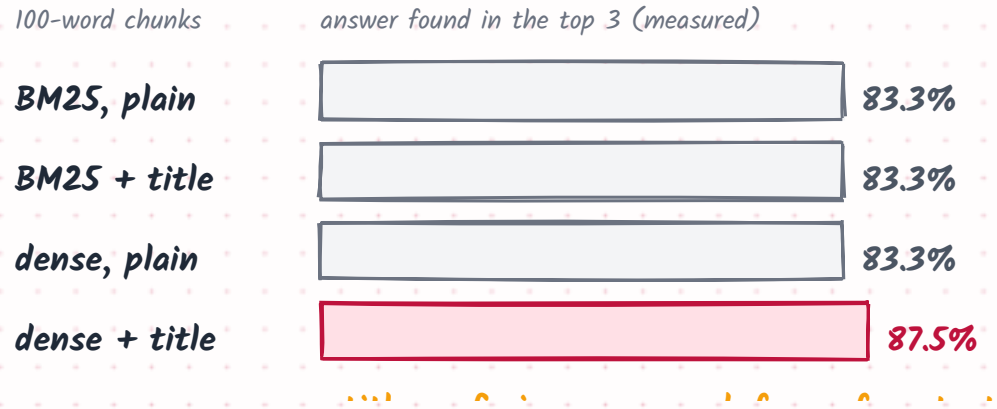
fusion needs methods that fail on DIFFERENT questions

*Top 3, 24 questions: both found it 21, only BM25 0, only dense 1, neither 2. Fusion helps when the methods fail on **different** questions; here they mostly agree. Each question = 4.2 points, so one-question gaps are noise. Larger benchmarks cited in the notebook show hybrid helping: verify on your own data.*

8 The two misses

- → "How much notice does AWS give before taking back a spot instance?": the word **spot** appears in many chunks, so dozens compete
- → "What value of k does RRF use?": the answer is symbols (k = 60), which my keyword tokenizer drops
- → Reading the misses is **error analysis**: it tells you what to fix

9 Does context help?



100-word chunks	plain hit@3	+ topic title hit@3
BM25	83.3%	83.3%
dense (stand-in)	83.3%	87.5%

A topic title is a very weak form of context. The real method (an LLM writes 1 to 2 sentences placing each chunk in its document) was reported to cut failed retrievals by 35% (embeddings), 49% (+ BM25) and 67% (+ reranking) in the original study (notebook). **Not reproduced here.** Cost about $1 per million document tokens, once, with prompt caching.

10 Seven failure points

Failure	Usual fix
content is missing	coverage checks; allow "I don't know"
right document ranked too low	hybrid, contextual chunks, query rewriting
found but dropped before the prompt	rerank, tune k
in the prompt but ignored	fewer, better-ordered chunks
wrong format	structured outputs
too vague or incomplete	decomposition + an eval set

11 RAG or one long prompt?



cached long prompt: over 2x lower latency, up to 90% cheaper (vendor)

Knowledge base under about **200K tokens** (about 500 pages), stable, one tenant: include it all and cache it (vendor: over 2x lower latency, up to 90% cheaper). Larger, changing, permissioned or multi-tenant: RAG. Cost per question grows with context, and models miss things in the middle.

12 Permissions and untrusted text

- ✓ Every chunk carries **who may read it**; filter **inside** the search, not afterwards
- ✓ A missing tenant filter leaks one customer's data into another's answer: the worst RAG bug
- ✓ Retrieved text is **data, not commands** (indirect prompt injection, lesson 10)

13 Evaluating RAG

- → Keep labelled questions with the passage that answers each
- → Measure **retrieval** (recall@k) apart from **answers** (is the final text right?)
- → Track both on every release

14 Beyond plain RAG

Idea	Good for	Cost
GraphRAG	big-picture questions ("main themes?")	a graph built at indexing time
Agentic RAG	multi-step questions	more calls and cost

15 Managed RAG on each cloud

Your data lives in	Look at
SharePoint / Microsoft 365	Azure AI Search + Foundry IQ
Amazon S3	Bedrock Knowledge Bases (S3 Vectors cheap at rest; OpenSearch Serverless for low-latency hybrid)
Google storage, BigQuery, Drive	Vertex AI RAG Engine / Vertex AI Search
minimal operations	OpenAI file search (data leaves your cloud)

All manage ingestion, chunking, embedding, storage, retrieval and grounded answers, and expose a retrieve-only call (notebook, Oct 2026).

16 ShopNest Copilot: RAG plan

Step	Plan
chunk	~300 words, overlap, metadata
index	embed once, record the model version
search	keywords + vectors, rerank
answer	cite sources, "not found" rule, filter by customer
test	40 real questions, recall@k per release

17 Words to keep

Word	Meaning
Chunk	a piece of a document, sized for search
Hybrid search	keywords and vectors together
Rerank	sort the shortlist with a smarter model
hit@k	share of questions whose answer is in the top k results

18 Interview corner

Q. How do you choose chunk size?

A. Start at 300 to 800 tokens with overlap, then measure on your own questions. In my test 300 words put the answer in the top 3 for 92% of questions, 800 words for 79%.

Q. Why hybrid search?

A. Vectors catch paraphrases, keywords catch exact terms; merge by reciprocal rank fusion. Verify on your data: in my test fusion did not beat the vector stand-in, because the methods failed on the same questions.

Q. When would you skip RAG?

A. When the whole knowledge base is under about 200K tokens, stable and single-tenant: cache it in the prompt.

Q. What is the worst RAG bug?

A. A missing tenant or permission filter that leaks one user's documents into another's answer.

19 Interview questions

"Walk me through a RAG pipeline."

Parse with layout, chunk, add context, embed once, hybrid search, rerank, then generate with chunk IDs: answer only from the sources, cite them, say 'not found' if absent.

Trap → Flattening tables when parsing. It gives wrong numbers.

"How does reciprocal rank fusion work?"

Score = Σ 1 ÷ (60 + rank) over the lists. It uses ranks only, so no score scaling is needed. A document ranked #1 by vectors and #3 by keywords scores 0.0323, beating #2 in one list (0.0161).

Trap → Adding raw scores from BM25 and cosine: they are on different scales.

"Your retrieval misses questions. How do you debug it?"

Read the misses. In my test 'spot' appeared in dozens of chunks, and 'k = 60' was symbols my tokenizer dropped. Then fix by type: hybrid search, contextual chunks, query rewriting, reranking, tuning k.

Trap → Swapping the embedding model before doing any error analysis.

"How do you evaluate a RAG system?"

Keep labelled questions with the passage that answers each. Measure retrieval (recall or hit@k) separately from the final answer, and track both on every release. In my test 300-word chunks put the answer in the top 3 for 91.7%.

Trap → Judging only the final answer, so you cannot tell retrieval failures from generation failures.

"Why use a reranker?"

Retrieve about 150 candidates cheaply, rerank with a cross-encoder and keep about 20. Too much context distracts: models skim the middle.

Trap → Stuffing every retrieved chunk into the prompt.

"RAG or a long-context prompt?"

A stable, single-tenant knowledge base under about 200K tokens: put it all in the prompt and cache it. Larger, changing, permissioned or multi-tenant: RAG.

Trap → Long context without caching: cost per question grows with the context.

20 Your turn

108_rag_eval.py, 108_context_prefix.py: replace the questions and the corpus with your own.

⚡ Revise in 60 seconds

- → RAG = open-book exam, 7 steps
- → 300 words: **91.7%** (dense) · 87.5% (BM25) in the top 3
- → 800 words: 87.5% · **79.2%**
- → fusion did not win: methods failed on the same questions
- → title prefix: 83.3% → 87.5% for dense, no change for BM25
- → under ~200K tokens: skip RAG, cache the prompt
- → filter by user; retrieved text is data
- → build your own eval set

Next lesson → CAI-09
GenAI apps: agents, MCP and reliability: tools, loops, protocols, retries and sandboxes
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**